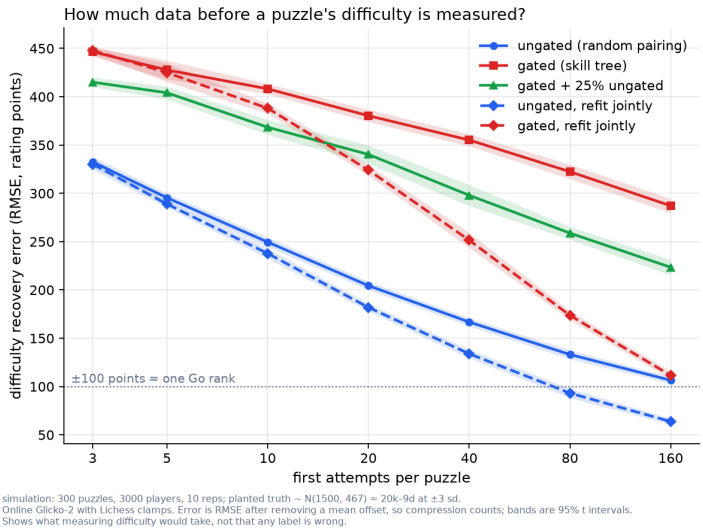
Measuring Go puzzle difficulty

How much attempt data does it take to measure a puzzle's difficulty, instead of assigning it by hand?

What I did. Plant 300 puzzles and 3,000 players with known difficulties and skills. Simulate their first-attempt logs under three exposure patterns; the one modelling Go Magic is skill-tree gating, where a player meets only puzzles near their own level. Fit each log twice with the truth hidden: online, one attempt at a time as a live system runs Glicko-2, and jointly, one fit over the whole log. Score both against the planted difficulties. Scale: **±100 points ≈ one Go rank** (EGF convention, accurate at 1d and generous below); assigning every puzzle the same difficulty scores 467.

> **About 160 first attempts measure a puzzle's difficulty to roughly one rank, but on skill-tree data only if the log is fitted *jointly*. Replaying the same log one attempt at a time, the standard online way, leaves almost three ranks of error: 287 points against 112, a 61% cut from how the fit is run alone.**
>
> The failure mode it avoids: reading 290 points of error off an online refit and concluding the data is inadequate.



Two exposure patterns, two estimators. Colour is the data: ungated random pairing against skill-tree gating as implied by Go Magic's public tree, green adding 25% ungated traffic. Line style is the estimator: solid online, dashed the same log refit jointly. Dotted line is one-rank accuracy; bands are 95% intervals over ten worlds.

Reproduce. `./src/recovery.py --puzzles 300 --reps 10` draws every curve here. Other numbers are one command each in `docs/RUNNING.md`; the estimator is checked against Glickman's worked example.

THREE RESULTS

Gating itself costs accuracy

Under gating no attempt directly compares an easy puzzle with a hard one: **1.6×** the error at 10 first attempts per puzzle, **2.7×** at 160, and a smaller check holds ~2.8× out to 1,280. Neither more traffic nor a better estimator closes it: refit jointly, a residual **1.75×** remains.

The damage is to the spacing, not the ordering

At 160 attempts the gated fit ranks puzzles well but compresses the scale to under half its width: it knows *which* puzzle is harder, not *by how much*. Compression is repairable.

Uneven traffic is a second, separate cost

Everybody attempts the first node; few reach the last. At matched volume that funnel costs **30–50** points and drops the ordering from 0.94 to **0.78**, the one thing gating had left intact.

at 160 first attempts	RMSE	fitted scale	ρ
ungated, online	107	1.20	0.99
ungated, joint	64	1.09	0.99
gated, online	287	2.36	0.94
gated, joint	**112**	**1.30**	1.00

RMSE in rating points, mean offset removed (±100 ≈ one rank). Fitted scale is the slope onto the planted truth: 1.0 is right, 2.36 means the spread is 2.36× too narrow. ρ is rank correlation, over ten worlds; within a regime both rows fit the identical log.

WHAT TO DO ABOUT IT

Backfill jointly, serve online

Fit the history once, jointly, for the labels; keep online Glicko-2 for the live path, where one-at-a-time updating is right.

Ship labels per puzzle, not per catalogue

Coverage will never be uniform: publish a measured label only where the attempts exist. But not on Glicko's own RD, which under gating claims **3.7×** more precision than it delivers.

Anchor the scale with an ungated test

An ungated diagnostic spanning the rank range is the standard repair for a compressed scale: 25% of traffic through one buys back **57** points and much of the compression.

What this does not claim. Not that any hand-assigned label is wrong; testing that needs the private attempt log. It answers the prior question: if difficulty were measured from attempts, how much data would the answer need to mean anything? That depends only on the estimator and the *shape* of the data, so simulation settles it. No private data is used.

The next step is small: run the joint fit over one month of the real log, first attempts only, and check the recovery curve against it.